**Journal: REST API**

Mickole R. Dechavez
BSIT – 3B

**Role:** Developer (Full-stack)

**Specific Tasks Performed:**

My assigned tasks for this project were to develop the frontend-backend interaction and create an API that updates user data. To achieve this, I merged pre-existing APIs that were previously separated from `src/api.php` and reworked the API method handling. Then, I implemented the `update_user` API in `src/api.php`. Then, I improved `src/client.php`, `src/index.html`, and matched request parameters with the API contracts in `src/api.php`.

On top of that, I've also migrated the project from an XAMPP-only environment to a XAMPP/Docker environment by adding Docker runtime files and env-based DB config in `src/db.php`. With the migration, I've also added an API that checks the status of the app and database.

Overall, I standardized the API to provide the required functions and resolved compatibility and migration problems.

**Code Contributions:**

1. **API consolidation and method handling improvements (api.php):** I merged previously separate API endpoints into the centralized API controller and modified request handling from a global POST-only restriction to per-action method validation. I also added support for parsing JSON and form payloads for non-POST requests.
2. **User update functionality (api.php):** I implemented the update_user action that allows updating username and password fields with validation, username uniqueness checking, record verification using user_id, and password hashing before saving updates to the database.
3. **Client and admin interface API alignment (client.php, index.html):** I updated the client and admin interface files to match the updated API contracts and improved how requests and responses were handled between the frontend and backend.
4. **Database configuration improvements (db.php):** I modified the database connection setup to support environment-variable-based configuration with fallback values for local environments.
5. **Docker compatibility support (runtime configuration files):** I migrated the project from a XAMPP-only setup to support both XAMPP and Docker environments by adding the necessary container runtime configuration files.
6. **Healthcheck endpoint implementation (healthcheck.php):** I created a standalone endpoint that verifies application and database availability during testing and deployment.
7. **API standardization and compatibility fixes (multiple files):** I standardized API behavior across actions and resolved compatibility issues encountered during integration and environment migration.

8. **API reliability and compatibility improvements (client.php, index.html, api.php):**

   Implemented endpoint fallback handling, improved error handling for API responses, standardized empty-response behavior, added output escaping to prevent rendering issues, and ensured compatibility with legacy password formats during login.

**Challenges Encountered:**

1. **XAMPP environment issues:** During my initial development for this project, I constantly faced issues with XAMPP, where MySQL would not run at all.
2. **Migrating from XAMPP-only to a Docker-compatible environment:** Had difficulties during migration, which required adjusting configuration so the system works in both XAMPP and Docker without breaking any existing behavior.
3. **Replacing global POST-only enforcement with per-action method validation:** I had to refactor `api.php` without breaking existing API behavior.
4. **Matching frontend requests with updated API contracts:** During my first few interactions with our frontend (both client and admin), there were mismatches in configurations.

**How I Contributed to Solving the Problems:**

1. I checked configuration differences and ensured that the database is up and running. I changed the configuration in order to resolve these issues. This includes: changing the port of MySQL, manually starting/stopping the mysql.service, and reinstalling XAMPP. However, I knew these issues would come back, and it would also show up in my groupmate's development in due time, so I came into conclusion to migrate it from an XAMPP-only to an XAMPP/Docker environment.
2. Initially, I copy pasted the folder and made a simple Docker configuration; however, there were a few incompatibilities, and it may cause issues in the future. To address this, I modified the Docker configuration to run the same version of Apache and MySQL (which is actually MariaDB) in XAMPP. I've also made a health check endpoint that allows me to verify if the system is working properly.
3. I've noticed that api.php enforces the POST method globally, which is not ideal, because other requests may require different actions. To fix this problem, I refactored api.php to allow a per-action method and ensured each behavior remained consistent after the change.
4. There is an existing mismatch between frontend requests and backend contracts. In order to fix this, I aligned request parameters and improved response handling so both interfaces interacted correctly with the updated API.